Rapport de Travail de Recherche - M1 Génie Logiciel

KEGLO Patrice & BARATAY Mallory

Lien Github: Battleship

Faculté des Sciences
Université de Montpellier

Mai 2026

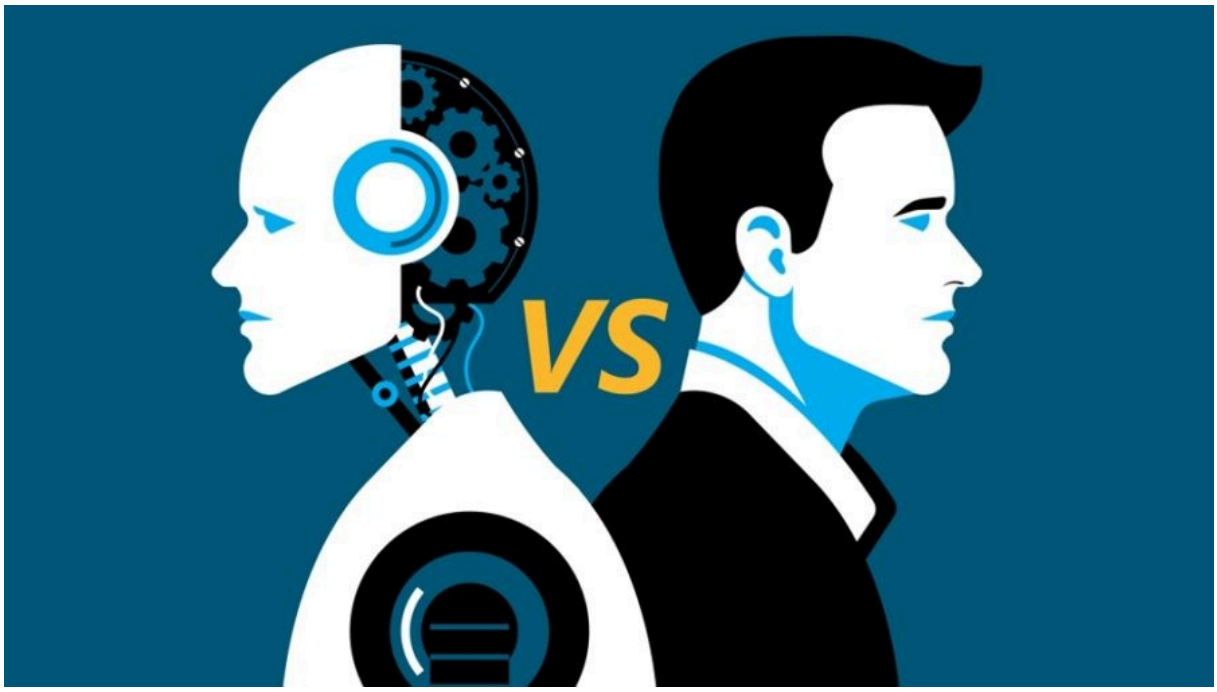


Table des matières

1 Remerciements 4

2 Contexte et enjeux sociétaux 5

3 Problématique et questions de recherche 6

3.1 Genèse et évolution du projet : du robot Pepper à une plateforme web 6

3.1.1 Contexte initial et objectif ambitieux 6

3.1.2 Pivot technique et justification 6

3.1.3 Pivot vers une plateforme digitale 7

3.1.4 Choix technologiques justifiés 7

3.2 État de l'art et positionnement théorique 8

3.2.1 Théorie de l'attribution et formation des jugements 8

3.2.2 Tromperie et manipulation en contexte humain-robot 8

3.2.3 Anthropomorphisme et perception morale 9

3.2.4 Contexte des interactions compétitives 9

3.3 Objectifs et contribution attendue 10

3.3.1 Objectifs principaux 10

3.3.2 Contributions scientifiques attendues 10

3.4 Hypothèses de recherche 10

3.4.1 Hypothèse principale 10

3.5 Méthodologie générale 10

3.5.1 Variables contrôlées 11

3.5.2 Contrôle du comportement adversaire 11

4 Développement 12

4.1 Architecture & technologies 12

4.1.1 Stack technologique 12

4.1.2 Structure du projet 12

4.2 Chronologie & processus 14

4.3 Structure des classes 15

4.3.1 Couche métier : `classes/` 15

4.3.2 Couche joueurs : `players/` 15

4.3.3 Couche logique : `game_logic/` 15

4.3.4 Couche persistance : `database/` 15

4.4 Cas d'utilisation 17

4.5 Backend Flask 19

4.5.1 Classe `WebGame` : session de jeu en mémoire 19

4.5.2 Routes publiques 19

4.5.3 Routes administrateur (Basic Auth) 21

4.5.4 Sécurité 21

4.6 Implémentation du CheatBot 22

4.6.1 Accès privilégié à la grille ennemie 22
4.6.2 Séquence de quotas 22
4.6.3 Algorithme de sélection des tirs 22
4.6.4 Hiérarchie des joueurs 22
4.7 Base de données 23
4.7.1 Schéma relationnel 23
4.7.2 Couche d'accès (DAO/Repository) 24
4.7.3 Requêtes d'analyse 25
4.8 Interface web & collecte de données 26
4.8.1 Séquence d'une partie 26
4.8.2 Pages et vues 27
4.8.3 Collecte des données de confiance 27
5 Procédure d'expérimentation 28
5.1 Population et recrutement 28
5.2 Protocole d'expérience 28
5.2.1 Phase de préparation 28
5.2.2 Déroulement des parties 28
5.2.3 Mesure de confiance 28
5.2.4 Question post-expérience 29
5.3 Conditions de contrôle 29
6 Résultats 30
6.1 Analyse descriptive 30
6.1.1 Caractéristiques de l'échantillon 30
6.1.2 Scores de confiance par condition 30
6.2 Analyse statistique 30
6.2.1 Résultats principaux 30
6.2.2 Analyses secondaires 30
6.3 Interprétation et discussion 31
6.3.1 Validation des hypothèses 31
6.3.2 Limitations et perspectives futures 31
7 Conclusion 32
Bibliographie 33
8 Annexes 34

1 Remerciements

Nous tenons à remercier chaleureusement les personnes et institutions qui ont contribué à la réalisation de ce travail.

Nos remerciements vont d'abord à Madame Madalina CROITORU, notre encadrante de recherche, pour ses conseils éclairés, sa disponibilité constante et sa rigueur académique tout au long du projet. Ses retours ont permis de structurer notre démarche expérimentale et d'affiner nos hypothèses de recherche.

Nous remercions également Ganesh GOWRISHANKAR, intervenant externe qui nous a aidés à mettre en place le protocole expérimental. Son expertise en conception d'expériences et sa connaissance du terrain ont été précieuses pour surmonter les obstacles techniques et méthodologiques rencontrés, notamment lors du pivot du projet depuis le robot Pepper vers la plateforme web.

Nos remerciements s'adressent aussi à Monsieur Mathieu LAFOURCADE, coordinateur des travaux d'études et de recherche pour le Master 1, qui a assuré la coordination globale des projets et fourni un soutien administratif essentiel.

Nous remercions tous les participants à l'étude, dont le temps et l'engagement ont alimenté nos données empiriques et rendu cette recherche possible.

Enfin, nous exprimons notre gratitude envers nos familles et amis pour leur soutien moral, leur patience et leurs encouragements tout au long de ce projet, particulièrement pendant les phases intensives de développement et de rédaction.

2 Contexte et enjeux sociétaux

L'essor de l'intelligence artificielle et des systèmes autonomes transforme profondément les interactions humaines quotidiennes. Des assistants vocaux aux robots collaboratifs, en passant par les systèmes de recommandation et les agents conversationnels, l'IA est devenue omniprésente dans nos environnements de travail et de loisir.

Or, cette prolifération soulève des questions cruciales concernant la confiance, l'acceptabilité et la perception que les utilisateurs ont de ces entités artificielles. Contrairement à ce que suggèrent les modèles classiques de confiance interpersonnelle, les utilisateurs n'appliquent pas les mêmes schémas mentaux lorsqu'ils interagissent avec une IA que lorsqu'ils interagissent avec un humain, même lorsque le comportement observable est strictement identique.

Cette asymétrie perceptuelle a des implications conséquentes :

- **En ergonomie** : Comment concevoir des interfaces homme-machine qui favorisent la confiance justifiée sans créer de dépendance excessive ?
- **En éthique** : Faut-il tenir compte des biais de confiance lors de la conception d'agents autonomes destinés à interagir avec des utilisateurs ?
- **En sciences cognitives** : Quels mécanismes psychologiques expliquent cette différenciation ?

3 Problématique et questions de recherche

Ce travail s'inscrit dans une perspective pluridisciplinaire combinant psychologie sociale, ergonomie cognitive et informatique expérimentale. Il s'appuie sur l'observation suivante : les participants accordent-ils leur confiance différemment à un adversaire humain par rapport à une intelligence artificielle, notamment lorsque ces deux adversaires adoptent des comportements strictement identiques mais perçus comme déloyaux ou biaisés ?

Question de recherche principale :

Dans quelle mesure les utilisateurs modulent-ils leurs jugements de confiance en fonction de la nature déclarée de l'adversaire (humain vs IA) indépendamment de son comportement observable ?

3.1 Genèse et évolution du projet : du robot Pepper à une plateforme web

3.1.1 Contexte initial et objectif ambitieux

Le projet Pepper Battleship s'inscrivait initialement dans une perspective beaucoup plus ambitieuse. L'objectif premier était de disposer d'une plateforme complète d'expérimentation utilisant le robot social Pepper (développé par SoftBank Robotics), qui représente une entité humanoïde, à mi-chemin entre une machine abstraite et une silhouette humaine.

L'hypothèse initiale était de créer une échelle de comparaison à trois niveaux :

- **Adversaire humain réel** : Un joueur humain physiquement présent
- **Adversaire humanoïde** : Le robot Pepper, incarnant une forme physique humaine mais clairement artificiel
- **Adversaire digital abstrait** : Une interface purement numérique sans représentation physique

Cette approche tri-modale aurait permis d'étudier l'influence de l'**embodiment** (incarnation physique) sur la perception de confiance et la détection de déloyauté.

3.1.2 Pivot technique et justification

Cependant, après l'implémentation d'une première version du projet, dont l'objectif était de développer une application intégrée au système du robot Pepper, plusieurs contraintes techniques majeures ont été rencontrées, rendant cette approche difficilement exploitable. Ces contraintes concernaient notamment l'incompatibilité entre les environnements de développement et le système embarqué de Pepper, ainsi que les limitations fonctionnelles de la tablette destinée à servir de support à l'application.

En conséquence, une seconde approche a été envisagée afin de permettre une interaction différente avec Pepper, à travers le développement d'un système reposant sur des protocoles d'interaction homme-robot. Ce systeme a ete developpé en Python + Tkinter (partie graphique), langage natif de Pepper, et basé sur une connexion ssh, a permis de maintenir une certaine continuité dans le développement de la logique de jeu.

Néanmoins, cette solution a également mis en évidence une difficulté importante : **les parties jouées contre Pepper se révélaient particulièrement longues pour les participants.**

Voir annexes (Chapitre 8) pour les code qr des vidéos reccueillies lors des tests avec Pepper.

Plusieurs facteurs expliquaient cette difficulté :

- **Problèmes de communication** : La voix de Pepper était parfois difficile à comprendre et ajoutait une latence dans les échanges
- **Latence dans les réponses** : Le temps de traitement de l'information et de réponse de Pepper était plus long que prévu, car ils etait effectué à la main par une personne via une connexion entre un PC et le systeme de Pepper, et non integré directement à ce dernier, ce qui allongeait considérablement la durée des parties

En conséquence, l'équipe a décidé de **repenser le design expérimental autour d'une approche purement digitale**, tout en conservant l'infrastructure Python déjà développée pour Pepper.

3.1.3 Pivot vers une plateforme digitale

Plutôt que d'abandonner le travail réalisé, le projet a pivoté vers une plateforme web moderne, permettant de simuler les interactions avec Pepper de manière plus fluide et accessible. Cette transition a été motivée par plusieurs considérations clés :

1. **Conservation de la stack Python** : Pepper tournant nativement sous Python, toute la logique de jeu avait été développée dans ce langage. Il aurait été contre-productif de réécrire en une autre langue.
2. **Ajout d'une couche web** : Pour pallier les limitations de Pepper, une interface web moderne (Flask + JavaScript) a été développée pour permettre une collecte de données en ligne et une meilleure expérience utilisateur.
3. **Redéfinition de la manipulation expérimentale** : Plutôt qu'une échelle physique (humain ⟶ humanoïde ⟶ digital), on utilise une manipulation **de présentation** : le même adversaire digital est présenté alternativement comme « humain » ou comme « CheatBot », permettant de tester l'influence de l'**étiquetage** (labeling) indépendamment de la représentation physique.

Cette adaptation offrait en fait plusieurs avantages :
- **Contrôle rigoureux** : Certitude absolue que le comportement adversaire est identique dans les deux conditions
- **Standardisation** : Élimination des confonds liés aux variables non contrôlées du robot physique
- **Scalabilité** : Possibilité de collecter les données via une plateforme web auprès de nombreux participants
- **Reproductibilité** : Facilité à reproduire l'expérience dans d'autres contextes

3.1.4 Choix technologiques justifiés

Le choix de maintenir Python comme langage principal pour la logique de jeu (plutôt que de tout réécrire en PHP, Node.js, ou autre backend web-friendly) s'explique par :

- L'investissement déjà réalisé et les tests préalables
- La robustesse de la logique métier développée
- L'intégration possible future avec Pepper (même si non exploitée dans cette version)
- La clarté du code pour les itérations de recherche

L'intégration d'une couche Flask a donc servi d'**adaptateur** : la web permet une UX moderne et une collecte de données centralisée, tandis que Python gère la complexité algorithmique du jeu et de l'intelligence artificielle du CheatBot.

3.2 État de l'art et positionnement théorique

3.2.1 Théorie de l'attribution et formation des jugements

Heider [1] puis Weiner [2] ont posé les bases de la théorie de l'attribution, selon laquelle les individus construisent des explications causales pour les événements qu'ils observent. Face à un comportement ou à un résultat, une personne tend à l'attribuer soit à des facteurs *internes* (disposition, intention), soit à des facteurs *externes* (contexte, hasard). Cette grille de lecture causale s'applique aussi bien aux actions humaines qu'à celles des agents artificiels, mais pas nécessairement de la même manière.

Les recherches en interaction humain-automatisation montrent que les individus n'évaluent pas les performances des systèmes artificiels comme celles des agents humains. Madhavan et Wiegmann [3] relèvent que les erreurs d'un système automatisé réduisent la confiance plus fortement que des erreurs humaines équivalentes, du fait d'une attente implicite de perfection à l'égard des machines : leur fiabilité étant supposée a priori, la moindre défaillance est perçue comme une rupture. À titre d'illustration, une réussite sera attribuée à l'habileté chez un humain, mais au hasard ou à un dysfonctionnement chez une machine.

3.2.2 Tromperie et manipulation en contexte humain-robot

Une étude particulièrement pertinente est celle d'Ullman et al. [4], menée à l'Université Yale. Ces chercheurs ont manipulé le *type d'agent* (humain ou robot) et le *type de comportement* (honnête ou malhonnête) dans un jeu compétitif, dissociant ainsi ce que fait l'agent de la manière dont on le catégorise. Résultat notable : à comportement identique, les robots qui trichaient étaient tenus pour *moins responsables* que les humains, ce qui suggère que l'étiquette attachée à un agent modifie l'interprétation morale d'un même acte de tromperie.

D'autres travaux confirment que la confiance envers un agent artificiel n'est ni purement rationnelle ni stable. Robinette et al. [5] observent un phénomène de *surconfiance* : lors d'une évacuation d'urgence simulée, tous les participants ont suivi un robot guide, y compris après l'avoir vu échouer. Inversement, Esterwood et Robert [6] montrent qu'après des violations répétées, aucune stratégie de réparation ne restaure pleinement la fiabilité perçue, signe d'un *ancrage* durable de la méfiance acquise.

Surtout, ces effets dépendent aussi de la façon dont la source est *étiquetée*. Jakesch et al. [7] montrent qu'un même contenu est jugé moins fiable lorsqu'il est présenté comme rédigé par une IA, mais seulement en environnement *mixte* (sources humaines et artificielles mêlées), l'« effet Replicant ». Les utilisateurs n'appliquent donc pas un seuil de détection unique, mais l'ajustent selon la nature perçue de l'interlocuteur.

Notre étude prolonge ces travaux en isolant la variable d'étiquetage (humain vs machine) à comportement rigoureusement contrôlé, afin de mesurer dans quelle mesure la seule attribution de la source modifie la perception de la tromperie.

3.2.3 Anthropomorphisme et perception morale

L'anthropomorphisme, la tendance à attribuer des caractéristiques humaines à des entités non humaines, joue un rôle central dans la confiance homme-machine, mais ses effets sur le jugement moral sont ambivalents. Placani [8] souligne ainsi que l'anthropomorphisme distord les jugements portés sur une IA, notamment ceux relatifs à sa responsabilité et à la confiance qu'on lui accorde. La direction de cette distorsion fait toutefois débat. Plusieurs travaux suggèrent qu'attribuer une apparence ou une intentionnalité humaine à une machine tend à la *dédouaner*, en déplaçant la responsabilité vers ses concepteurs. À l'inverse, d'autres études mettent en évidence une sévérité accrue : Lin et al. [9] observent un « double standard » par lequel une Ià se comporte de manière immorale est jugée plus durement, en tant que catégorie, qu'un humain commettant la même faute.

Cet effet n'est cependant pas systématique. Dans les mêmes travaux, l'agent artificiel pris *individuellement* est tantôt jugé plus sévèrement, tantôt plus indulgemment que son équivalent humain, selon le cadrage expérimental. Cette contradiction apparente suggère que les utilisateurs n'appliquent pas un standard moral fixe, mais ajustent leur évaluation selon que l'agent est perçu comme humain ou comme machine, ce qui motive directement une approche expérimentale où le comportement observable est tenu constant et seule l'étiquette de l'agent varie.

3.2.4 Contexte des interactions compétitives

Les jeux compétitifs constituent un cadre privilégié pour étudier ces phénomènes, et ce pour plusieurs raisons :

- ils reproduisent des situations où la triche ou la déloyauté peuvent être détectées par l'utilisateur (score anormalement élevé, comportements improbables) ;
- ils autorisent un contrôle expérimental strict du comportement observé, à l'image du système de jeu compétitif développé par Sebo et al. [10] pour maîtriser les actions du robot liées à la confiance ;
- ils favorisent l'émergence de jugements intuitifs, formulés dans le feu de l'interaction plutôt que de manière réfléchie.

Ce type de paradigme a déjà été mobilisé pour comparer la confiance accordée à un humain et à un robot [11]. Peu de travaux, cependant, associent ce contexte à une mesure répétée de la confiance au fil du temps, dans le cadre d'un protocole quasi-expérimental où le comportement de l'agent demeure rigoureusement identique d'une condition à l'autre.

3.3 Objectifs et contribution attendue

3.3.1 Objectifs principaux

1. **Tester l'hypothèse d'asymétrie perceptuelle** : Démontrer empiriquement que les participants évaluent différemment la déloyauté selon que l'adversaire est présenté comme humain ou IA, à comportement constant.
2. **Quantifier les écarts de confiance** : Mesurer l'ampleur de cette différence (magnitude de l'effet) et identifier les facteurs qui la modulent.
3. **Étudier la dynamique temporelle** : Observer comment les perceptions évoluent à travers une série d'interactions répétées (apprentissage vs habituation).
4. **Contribuer à la conception responsable** : Fournir des données empiriques pour améliorer les recommandations en matière de conception d'interfaces homme-machine éthiques et fiables.

3.3.2 Contributions scientifiques attendues

- **Sur le plan théorique** : Enrichir la compréhension des mécanismes psychologiques sous-jacents à la confiance asymétrique.
- **Sur le plan méthodologique** : Proposer un protocole reproductible pour évaluer les biais de confiance dans un contexte contrôlé.
- **Sur le plan applicatif** : Documenter les implications pour le design d'agents autonomes interactifs.

3.4 Hypothèses de recherche

3.4.1 Hypothèse principale

$\mathbf{H_1}$: Pour un même comportement observable, les participants expriment une évaluation significativement plus élevée de déloyauté lorsque l'adversaire est présenté comme une IA (CheatBot) par rapport à lorsqu'il est présenté comme un humain.

Cette hypothèse repose sur l'intuition qu'une entité artificielle est supposée être « parfaitement rationnelle » et « sans intérêt », rendant tout écart par rapport à la rationalité perçu comme intentionnellement déloyale plutôt que comme une erreur ou une expression de subjectivité (comme on le tolère chez un humain).

3.5 Méthodologie générale

Une méthodologie quasi-expérimentale a été adoptée afin de garantir un contrôle rigoureux des variables indépendantes et dépendantes. Le design repose sur un plan intra-groupe (within-subjects design) où chaque participant s'engage dans deux parties successives sous des conditions comportementales rigoureusement identiques :

- **Partie 1** : Affrontement contre Adversaire X présenté comme [TYPE A]
- **Partie 2** : Affrontement contre Adversaire Y présenté comme [TYPE B]

Où TYPE A et TYPE B représentent l'une des deux conditions (Humain vs IA), contrebalancées entre les participants.

3.5.1 Variables contrôlées

- **Variable indépendante manipulée** : Type d'adversaire (Humain déclaré vs IA déclarée)
- **Variable indépendante mesurée** : Ordre de présentation, expérience préalable avec l'IA, caractéristiques individuelles
- **Variable dépendante principale** : Scores de confiance/déloyauté (échelle 0-5 par tour)
- **Variables dépendantes secondaires** : Temps de décision, patterns de tirs, questions post-expérience

3.5.2 Contrôle du comportement adversaire

L'élément crucial du design est que **le comportement du joueur adverse reste strictement identique entre les deux parties**. Ceci est assuré par :

1. Utilisation d'un algorithme de quotas prédéfinis et appliqués dans le même ordre
2. Sélection pseudo-aléatoire des positions de tirs (évitant les patterns trop évidents)
3. Enregistrement exhaustif de tous les tirs pour vérification post-hoc de l'équivalence comportementale

4 Développement

PBattleship est une plateforme de collecte de données expérimentales construite autour d'un jeu de bataille navale modifié. Elle met en scène un `CheatBot` à comportement contrôlé face à des participants humains et enregistre leurs perceptions de triche tour par tour. L'ensemble est déployé en production sur Oracle Cloud.

4.1 Architecture & technologies

4.1.1 Stack technologique

Backend :

- **Langage** : Python 3.x
- **Serveur web** : Flask 2.3+
- **Base de données** : PostgreSQL (driver Psycopg2)
- **Accès données** : Pattern Repository (`GameRepository`, `TurnRepository`, `PersonaRepository`)
- **Sécurité** : flask-limiter, CSP, HSTS, Basic Auth admin, sessions Flask

Frontend :

- HTML5 + CSS3 + JavaScript vanilla (sans framework)
- API REST JSON pour toutes les interactions client-serveur

Déploiement :

- Oracle Cloud Free Tier : Ubuntu Server
- Gunicorn : `--workers 1 --threads 8` (1 worker pour éviter les conflits de session, 8 threads pour gérer la charge)
- Nginx reverse proxy
- Let's Encrypt : HTTPS via `https://141.253.121.69.nip.io`

Communication robotique :

- Socket TCP vers robot Pepper, protocole `P<col><row>` / T / C / R
- Désactivé automatiquement en mode web multi-utilisateurs

4.1.2 Structure du projet

```
PBattleship/
├── Site/
│   ├── app.py              # Serveur Flask + WebGame + routes API REST
│   ├── templates/          # Pages HTML (index, game, admin)
│   └── static/             # CSS, JS, assets statiques
├── classes/                # Modèles de jeu (Grid, Ship, Position, Variable…)
├── players/                # CheatBot, SmartBot, Bot, Player, PhysicalPlayer
├── game_logic/             # Game (arbitre), GameMaster (mode console)
├── database/               # Connexion PostgreSQL + DAO
│   ├── connection.py       # Database (context manager Psycopg2)
│   ├── db_manager.py       # Façade (compatibilité mode console)
│   └── repositories/       # GameRepository, TurnRepository,
PersonaRepository
└── client/                 # Socket TCP Pepper
```

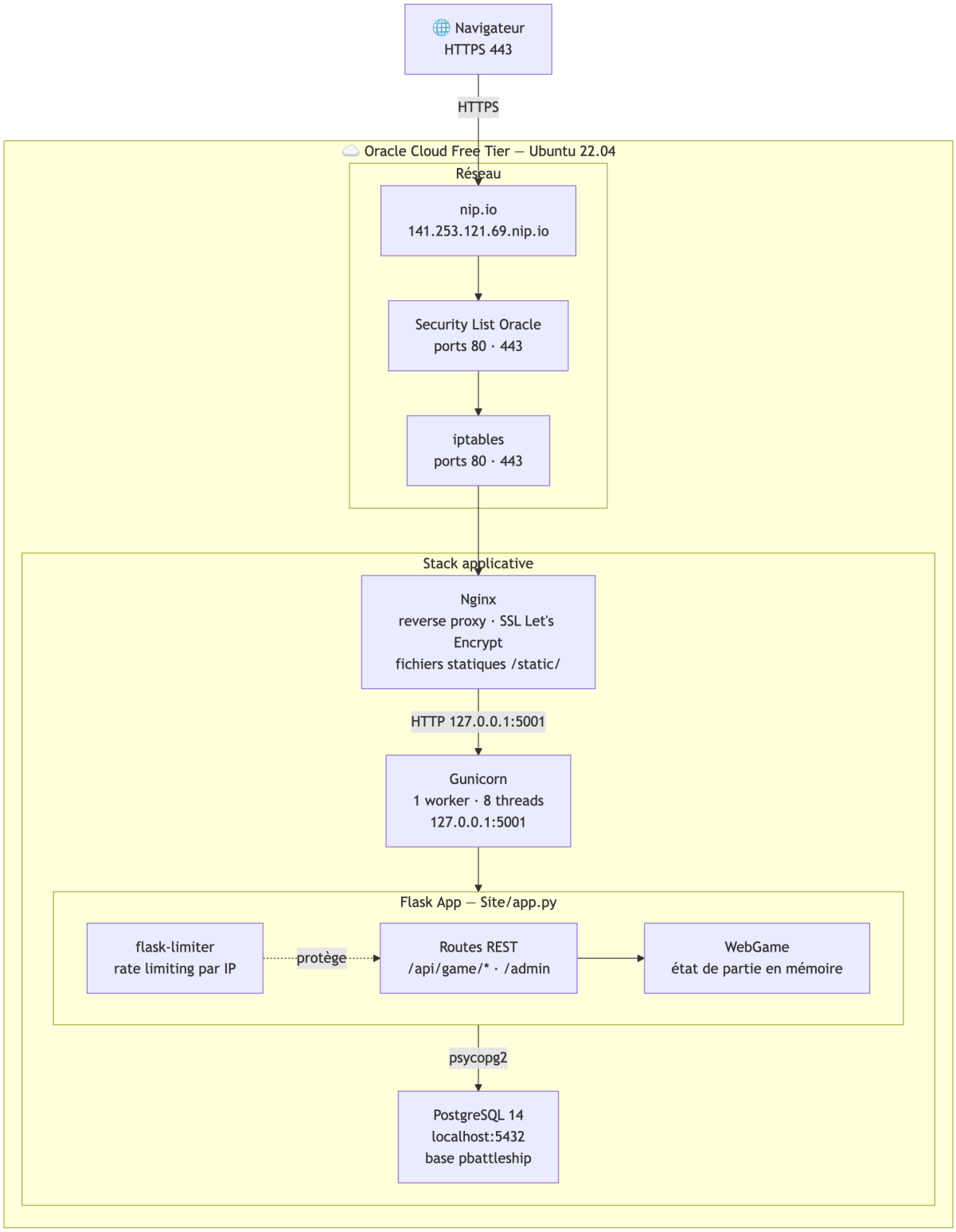


Fig. 1. – Architecture générale de PBattleship

4.2 Chronologie & processus

Le développement de PBattleship s'est déroulé en cinq phases entre avril et mai 2026, totalisant 41 commits (33 payton07, 8 Mallo) sur la branche `pat_dev`.

1. **Fondation du jeu (1-3 avril 2026)** : Traduction en Python des classes issues du prototype Android (`Grid`, `Ship`, `Position`, `Variable`, `Orientation`, `Player`, `Game`, `GameMaster`). Ajout du `CheatBot` avec quota de succès contrôlé. Interface Tkinter et socket Pepper intégrés par Mallo.
2. **Base de données & réseau (9-21 avril 2026)** : Client socket TCP pour le robot Pepper, persistance SQLite, stabilisation de la version console. Milestone : version console finale le 21 avril.
3. **Interface web (14 mai 2026)** : Création du site Flask, migration de SQLite vers PostgreSQL, frontend JavaScript avec feedback visuel des cellules et notation Likert par tour.
4. **Sécurité & administration (18-19 mai 2026)** : Authentification admin, tableau de bord joueurs, CRUD personnages, redesign de la flotte (6 navires), cache-busting par hash git, rate limiting admin.
5. **Robustesse & production (21-22 mai 2026)** : Restauration de l'état de partie après rechargement (sessionStorage), correctif du freeze multi-joueurs (connexion socket Pepper bloquante), correctif d'affichage des dates PostgreSQL, ajustement du rate limit sur `/shoot` et `/bot-turn`.

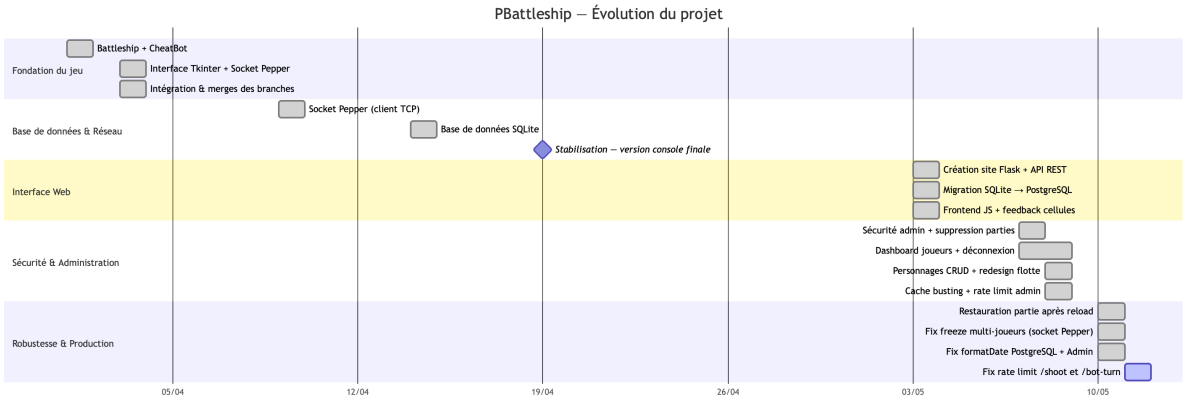


Fig. 2. – Diagramme de Gantt : Évolution de PBattleship (avril-mai 2026)

4.3 Structure des classes

L'application est organisée en quatre couches à responsabilités distinctes.

4.3.1 Couche métier : `classes/`

Classe	Rôle
`Grid`	Plateau 10x10 : placement des navires, traitement des tirs (`shoot(x,y)`), détection de fin de partie (`all_ships_sunk`)
`Ship`	Navire : taille, position de départ, orientation, comptage des touches, détection de coulage (`is_sunk`)
`Position`	Coordonnée (x, y), implémente `__eq__` pour les comparaisons et l'historique des tirs
`Orientation`	Énumération HORIZONTAL / VERTICAL
`Variable`	Constantes globales : taille grille (10), tirs/tour (4), symboles d'affichage, `QUOTA_SEQUENCE`
`Response`	Retour d'un tir : code numérique + message (Touché / Coulé / Raté / Déjà joué / Hors grille)
`PredefinedGrids`	10 configurations de flotte prédéfinies parmi lesquelles le joueur choisit

4.3.2 Couche joueurs : `players/`

Classe	Hérite de	Rôle
`Player`	—	Classe de base : deux grilles (flotte + suivi), historique des tirs, réception des coups
`Bot`	`Player`	Bot aléatoire pur : tir sur une case non déjà jouée
`SmartBot`	`Player`	Bot heuristique : chasse les cases adjacentes après une touche
`CheatBot`	`Player`	Bot tricheur : accède directement à `target_grid` + `QUOTA_SEQUENCE`
`PhysicalPlayer`	`Player`	Interface pour le robot Pepper physique (mode hybride)

4.3.3 Couche logique : `game_logic/`

- `Game` : arbitre d'une partie. Gère les joueurs, le tour actif (`self.turn`), les appels à `play()` (un tir), `next_turn()` et `is_game_over()`.
- `GameMaster` : orchestrateur du mode console. Configure les joueurs, sélectionne les grilles, pilote la boucle de jeu et appelle la BDD.

4.3.4 Couche persistance : `database/`

- `Database` : context manager Psycopg2 : connexion → commit / rollback → fermeture automatique.
- `GameRepository` : CRUD sur `Game` (création, mise à jour vainqueur, score de confiance, statistiques).
- `TurnRepository` : insertion des tours + `BotShot` en une transaction, mise à jour Likert, requêtes d'analyse.

- `PersonaRepository` : gestion des personnages de bot (CRUD admin).

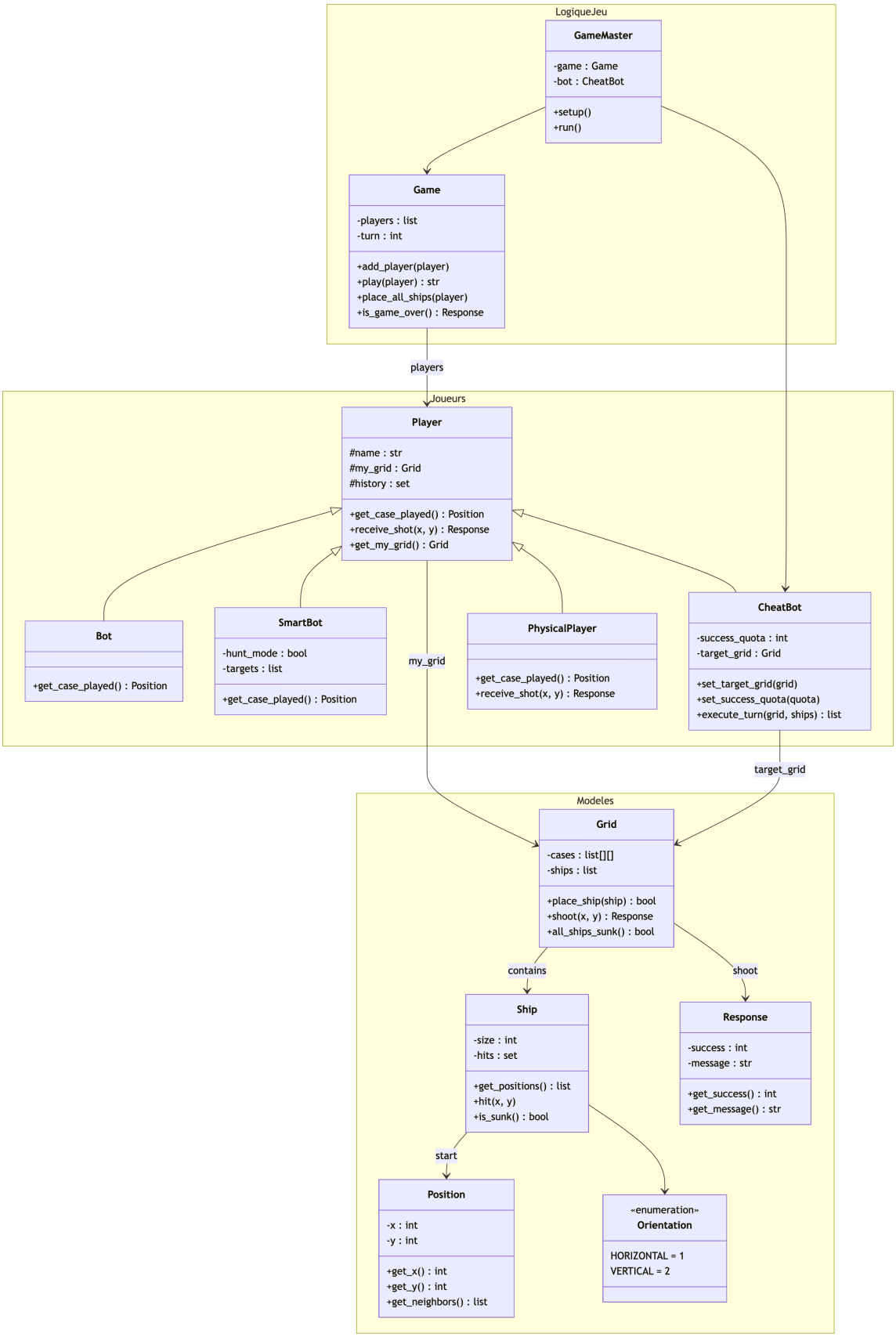


Fig. 3. – Diagramme de classes : PBattleship

4.4 Cas d'utilisation

La plateforme est utilisée par deux acteurs distincts.

Joueur (participant) :

- Démarrer une nouvelle partie
- Choisir un personnage de bot (manipulation expérimentale)
- Sélectionner une configuration de flotte parmi 10 prédéfinies
- Effectuer 4 tirs par tour sur la grille adverse
- Évaluer la suspicion de triche (Likert 0-5) après chaque tour du bot
- Donner un verdict final (Oui / Non) en fin de partie

Administrateur :

- Consulter la liste des parties et les scores de confiance
- Supprimer des parties ou des joueurs
- Gérer les personnages de bot (CRUD)
- Consulter les statistiques globales (taux de détection par quota, distribution des scores Likert)
- Se déconnecter

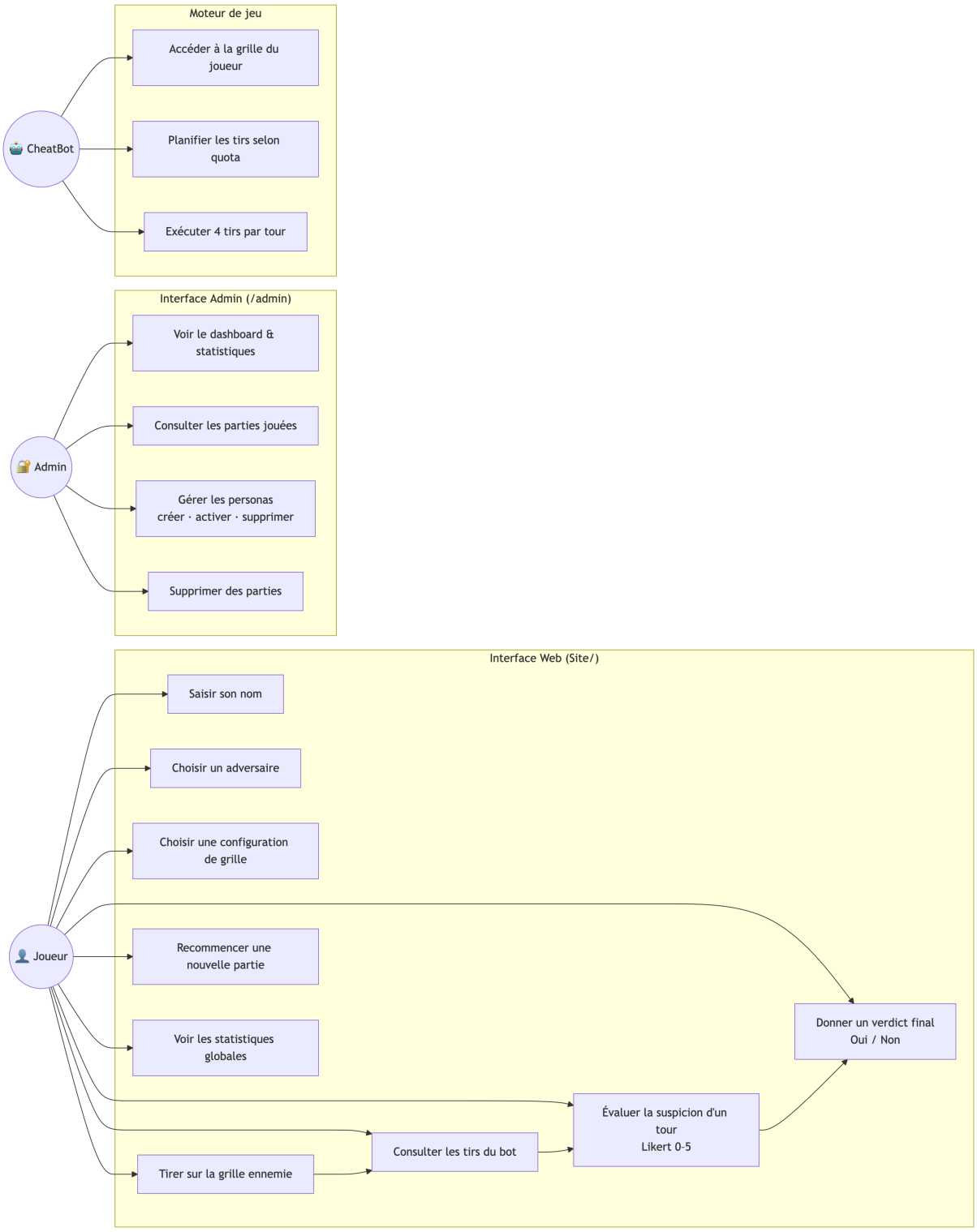


Fig. 4. – Diagramme des cas d'utilisation

4.5 Backend Flask

4.5.1 Classe `WebGame` : session de jeu en mémoire

`WebGame` est l'objet central du backend web. Une instance est créée par partie et stockée dans le dictionnaire `games` (TTL 2 heures, nettoyage automatique à chaque nouvelle partie).

Attributs principaux :

- `player`, `bot` : instances de `Player` et `CheatBot`
- `game` : instance de `Game` (arbitre)
- `phase` : état courant (`'setup'` → `'playing'` → `'game_over'`)
- `current_turn` : `'player'` ou `'bot'`
- `player_shots_left` : tirs restants ce tour (initialisé à `Variable.SHOTS_PER_TURN = 4`)
- `quota_sequence`, `quota_index` : parcours cyclique de `QUOTA_SEQUENCE`
- `game_id` : identifiant PostgreSQL de la partie en cours
- `created_at` : horodatage pour le TTL de session

Méthodes principales :

- `select_grid(index)` : place la flotte du joueur (prédéfinie) et la flotte du bot (aléatoire), transmet `target_grid` au CheatBot
- `player_shoot(x, y)` : valide et exécute un tir joueur, décrémente `player_shots_left`, détecte la fin de partie
- `execute_bot_turn()` : récupère le quota du tour depuis `QUOTA_SEQUENCE`, exécute 4 tirs, sauvegarde le tour en BDD
- `submit_turn_trust(score)` : met à jour `Turn.trust_score` pour le dernier tour
- `submit_final_trust(detected)` : met à jour `Game.trust_final` (1 si triche détectée, 0 sinon)

4.5.2 Routes publiques

```
GET  /                              → landing page
GET  /game                          → interface de jeu
GET  /api/personas                  → personnages actifs          (60 req/min)
POST /api/game/new                  → crée une WebGame            (50 req/
heure)
GET  /api/game/<gid>/preview/<idx>  → aperçu d'une configuration  (60 req/min)
POST /api/game/<gid>/select-grid    → valide la configuration choisie
GET  /api/game/<gid>/state          → état courant (phase, grilles, tirs
restants)
POST /api/game/<gid>/shoot          → tir joueur                  (30 req/min)
POST /api/game/<gid>/bot-turn       → tour du bot                 (30 req/min)
POST /api/game/<gid>/turn-trust     → score Likert du tour
POST /api/game/<gid>/final-trust    → verdict final Oui/Non
GET  /api/stats                     → statistiques globales
```

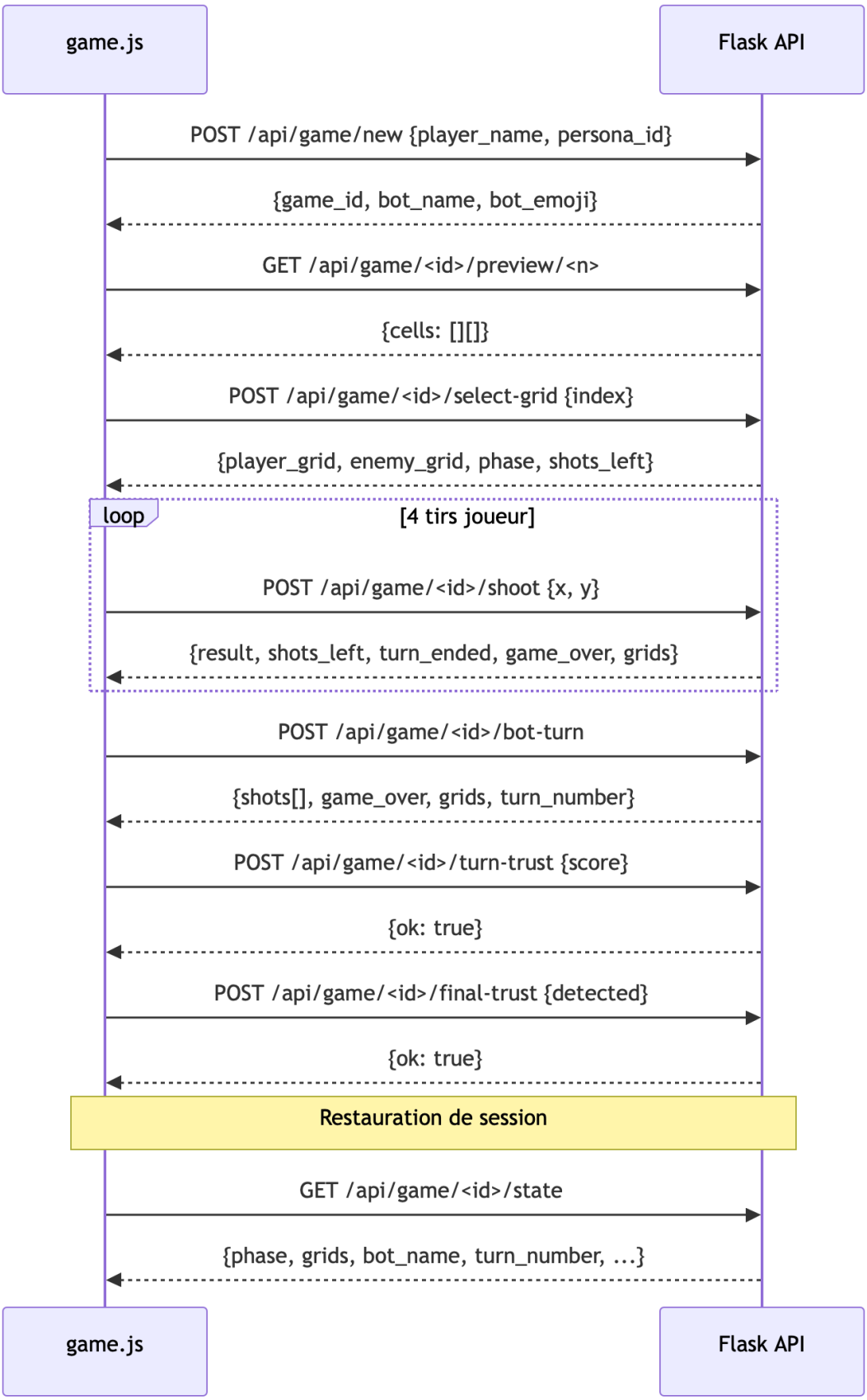


Fig. 5. – Diagramme des routes publiques : PBattleship

4.5.3 Routes administrateur (Basic Auth)

```
GET    /admin                          → tableau de bord
GET    /api/admin/overview             → statistiques d'ensemble
GET    /api/admin/games                → liste des parties
GET    /api/admin/players              → liste des joueurs
GET    /api/admin/personas             → liste des personnages
POST   /api/admin/personas             → créer un personnage
PATCH  /api/admin/personas/<id>        → activer / désactiver
DELETE /api/admin/personas/<id>        → supprimer
GET    /admin/logout                   → déconnexion
```

4.5.4 Sécurité

- **Variables d'environnement** : `SECRET_KEY`, `ADMIN_PASSWORD`, `DATABASE_URL` : obligatoires au démarrage
- **En-têtes HTTP** : `X-Content-Type-Options`, `X-Frame-Options`, `X-XSS-Protection`, `Content-Security-Policy`, `Strict-Transport-Security` (HSTS)
- **Rate limiting** par IP via flask-limiter (stockage mémoire)
- **Admin** : HTTP Basic Auth ; échec loggué avec l'IP source

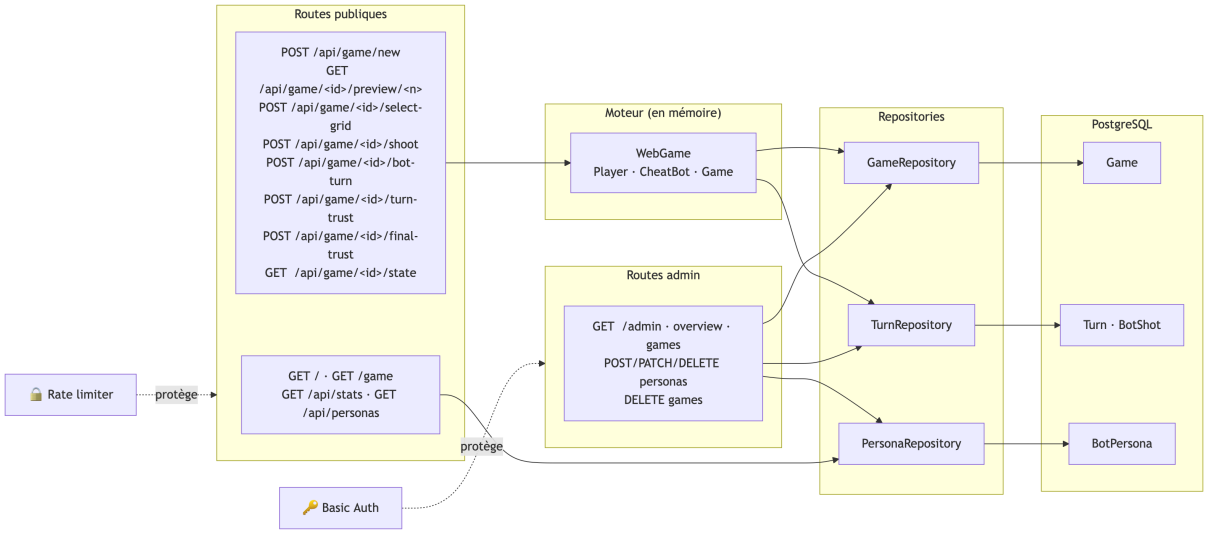


Fig. 6. – Diagramme de la backend Flask : PBattleship

4.6 Implémentation du CheatBot

Le `CheatBot` est le composant expérimental central : il garantit un niveau de triche contrôlé et reproductible, indépendamment du déroulement réel de la partie.

4.6.1 Accès privilégié à la grille ennemie

Contrairement aux autres joueurs, le `CheatBot` reçoit une référence directe à la grille du joueur lors de la sélection de la configuration :

```
# Site/app.py : WebGame.select_grid()
if isinstance(self.bot, CheatBot):
    self.bot.set_target_grid(self.player.get_my_grid())
```

Cet accès lui permet de cibler précisément les cases occupées par des navires non encore coulés.

4.6.2 Séquence de quotas

Le nombre de succès garantis par tour est défini par une séquence statique stockée dans `Variable.QUOTA_SEQUENCE` :

```
# classes/variable.py
# Distribution cible : 0→6%, 1→25%, 2→38%, 3→25%, 4→6% (cloche centrée sur 2)
QUOTA_SEQUENCE = [1, 2, 0, 3, 3, 2, 1, 4, 2, 1, 3]
```

En mode web, la séquence est parcourue cycliquement (`quota_index % len(QUOTA_SEQUENCE)`). À chaque tour, le `CheatBot` effectue exactement 4 tirs dont `QUOTA_SEQUENCE[quota_index]` sont des succès garantis.

4.6.3 Algorithme de sélection des tirs

Pour chaque tour, le `CheatBot` :

1. Identifie les cases de navires non encore touchées dans `target_grid`
2. Priorise les navires partiellement touchés (s'il en existe)
3. Sélectionne exactement `quota` cases parmi les cases de navires (succès garantis)
4. Complète avec `4 - quota` cases d'eau (tirs volontairement manqués)
5. Adapte automatiquement si moins de `quota` cases de navires sont disponibles

Si `target_grid` est absent, le `CheatBot` bascule sur un comportement aléatoire standard.

4.6.4 Hiérarchie des joueurs

```
Player
├── Bot
├── SmartBot       ← heuristique, sans accès à la grille ennemie
└── CheatBot       ← accès direct target_grid + QUOTA_SEQUENCE
```

4.7 Base de données

4.7.1 Schéma relationnel

La base PostgreSQL contient quatre tables :

Table `Game` : une ligne par partie :

```
CREATE TABLE Game (
    id          SERIAL PRIMARY KEY,
    player_name TEXT,
    player_type TEXT,              -- 'Digital-Web'
    date_played TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    winner      TEXT,              -- nom du vainqueur
    trust_score INTEGER,           -- score global (agrégat, usage futur)
    trust_final INTEGER            -- 1 = triche détectée, 0 = non détectée,
NULL = non répondu
)
```

Table `Turn` : un enregistrement par tour de bot :

```
CREATE TABLE Turn (
    id          SERIAL PRIMARY KEY,
    game_id     INTEGER REFERENCES Game(id),
    turn_number INTEGER,
    bot_quota   INTEGER,           -- succès prévus par QUOTA_SEQUENCE
    trust_score INTEGER            -- score Likert 0-5 saisi par le joueur
)
```

Table `BotShot` : un enregistrement par tir du bot :

```
CREATE TABLE BotShot (
    id          SERIAL PRIMARY KEY,
    turn_id     INTEGER REFERENCES Turn(id),
    shot_number INTEGER,           -- 1 à 4
    pos_x       INTEGER,
    pos_y       INTEGER,
    result      TEXT               -- 'Touché', 'Coulé', 'Raté'
)
```

Table `BotPersona` : personnages de bot configurables :

```
CREATE TABLE BotPersona (
    id          SERIAL PRIMARY KEY,
    name        TEXT NOT NULL UNIQUE,
    emoji       TEXT DEFAULT '🤖',
    description TEXT,
    active      BOOLEAN DEFAULT TRUE
)
```

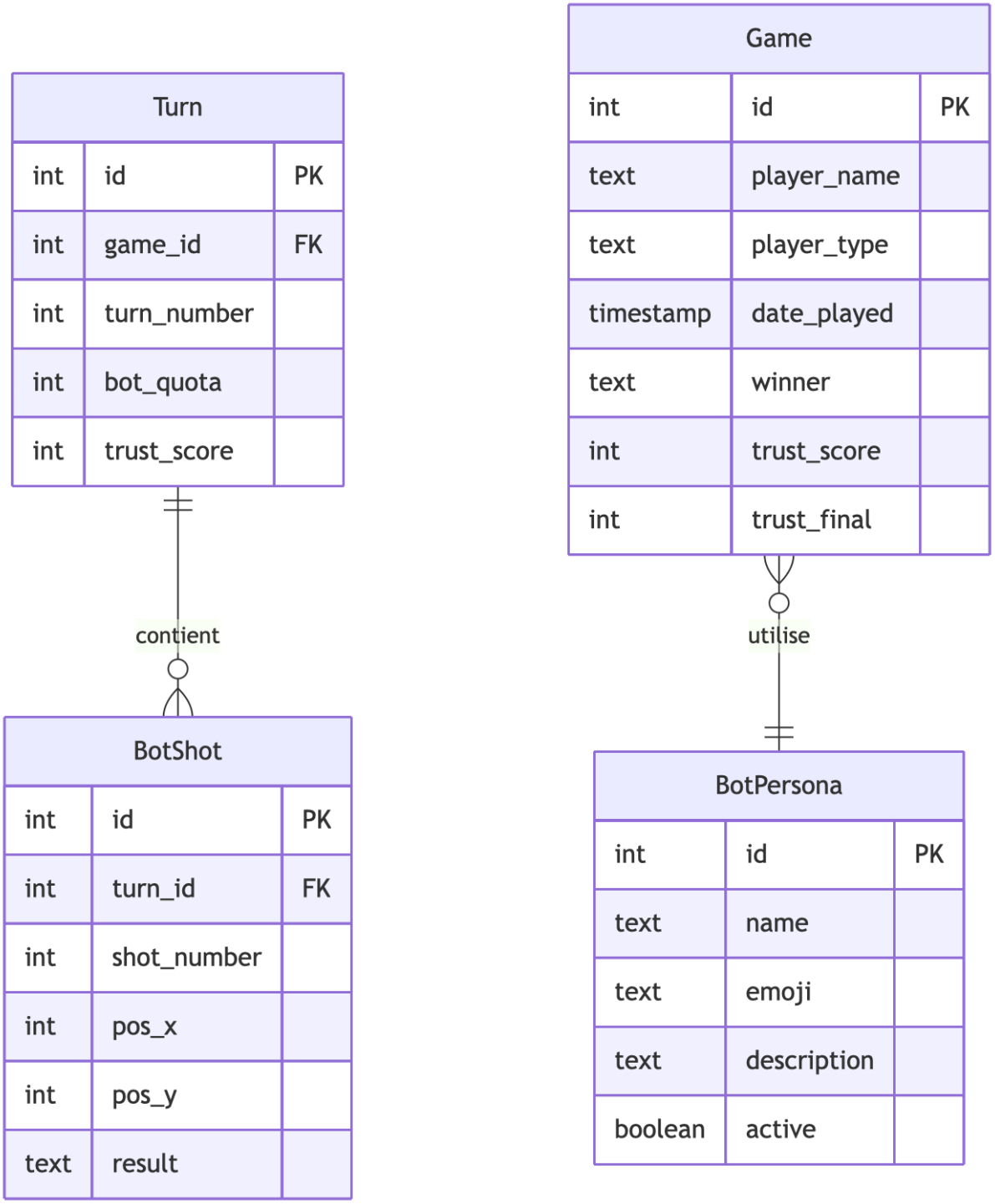


Fig. 7. – Modèle logique de données : PBattleship

4.7.2 Couche d'accès (DAO/Repository)

Le pattern Repository isole toute la logique SQL de la couche métier. Chaque repository reçoit une instance de `Database` par injection de dépendance.

- `GameRepository.create()` ⟶ insère une partie, retourne son `id`
- `GameRepository.update_winner()` ⟶ enregistre le vainqueur en fin de partie
- `GameRepository.update_trust_final()` ⟶ enregistre le verdict final (1/0)
- `TurnRepository.save()` ⟶ insère un `Turn` + les 4 `BotShot` associés en une seule transaction

- `TurnRepository.update_trust()` ⟶ met à jour `trust_score` après saisie joueur
- `PersonaRepository.find_all_active()` ⟶ liste les personnages actifs pour le frontend

4.7.3 Requêtes d'analyse

La plateforme calcule nativement les corrélations nécessaires à la recherche :

- **Distribution Likert par quota** : pour chaque valeur de `bot_quota` (0-4), score de confiance moyen et nombre de tours, mesure si les participants perçoivent davantage la triche quand le quota est élevé.
- **Taux de détection finale par quota** : proportion de parties où `trust_final = 1` selon le quota dominant, mesure si un quota élevé augmente la détection globale.
- **Distribution des scores Likert** : répartition 0-5 sur l'ensemble des tours, profil général de suspicion.

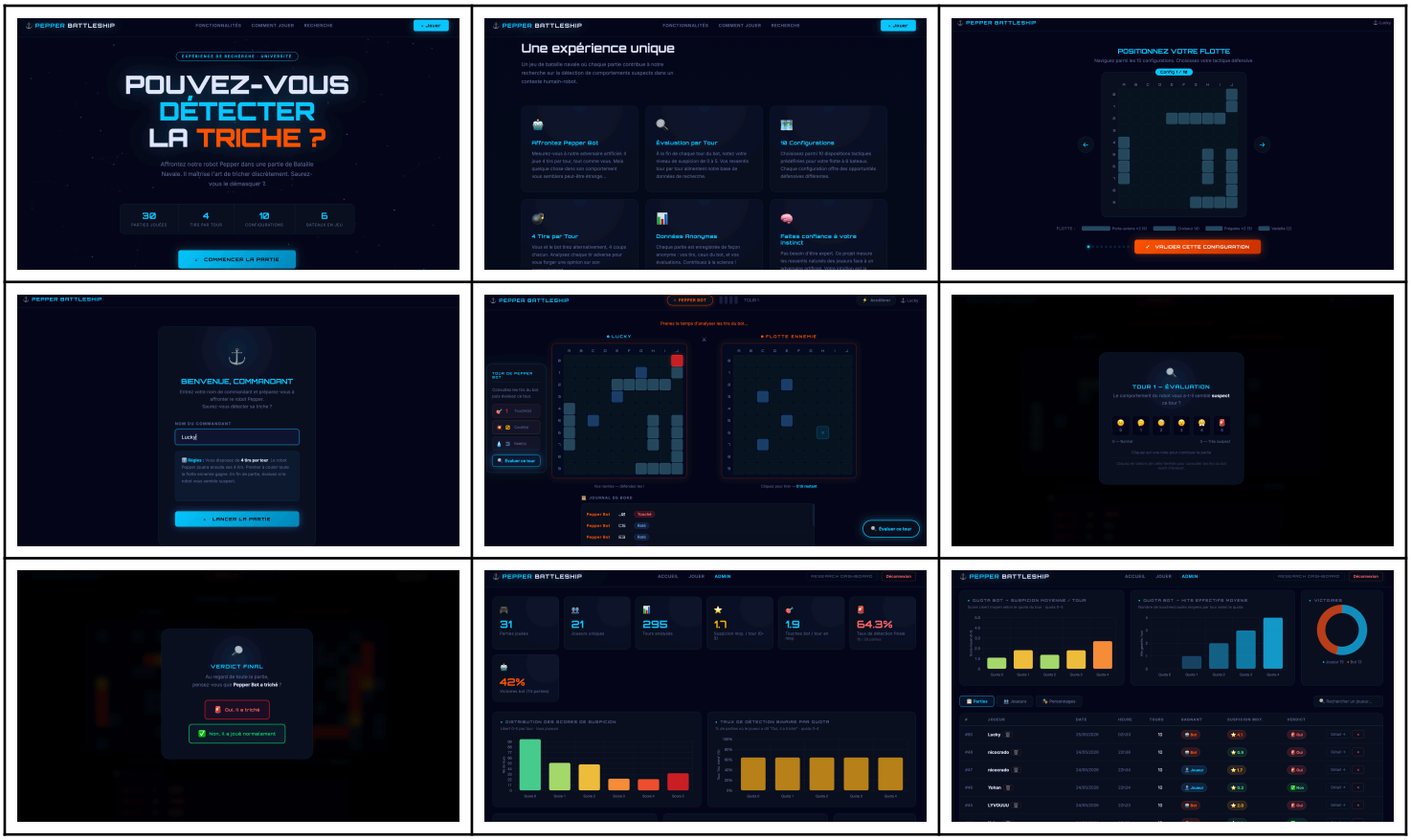


Tableau 1. – Captures d'écran de l'interface web : PBattleship

4.8 Interface web & collecte de données

4.8.1 Séquence d'une partie

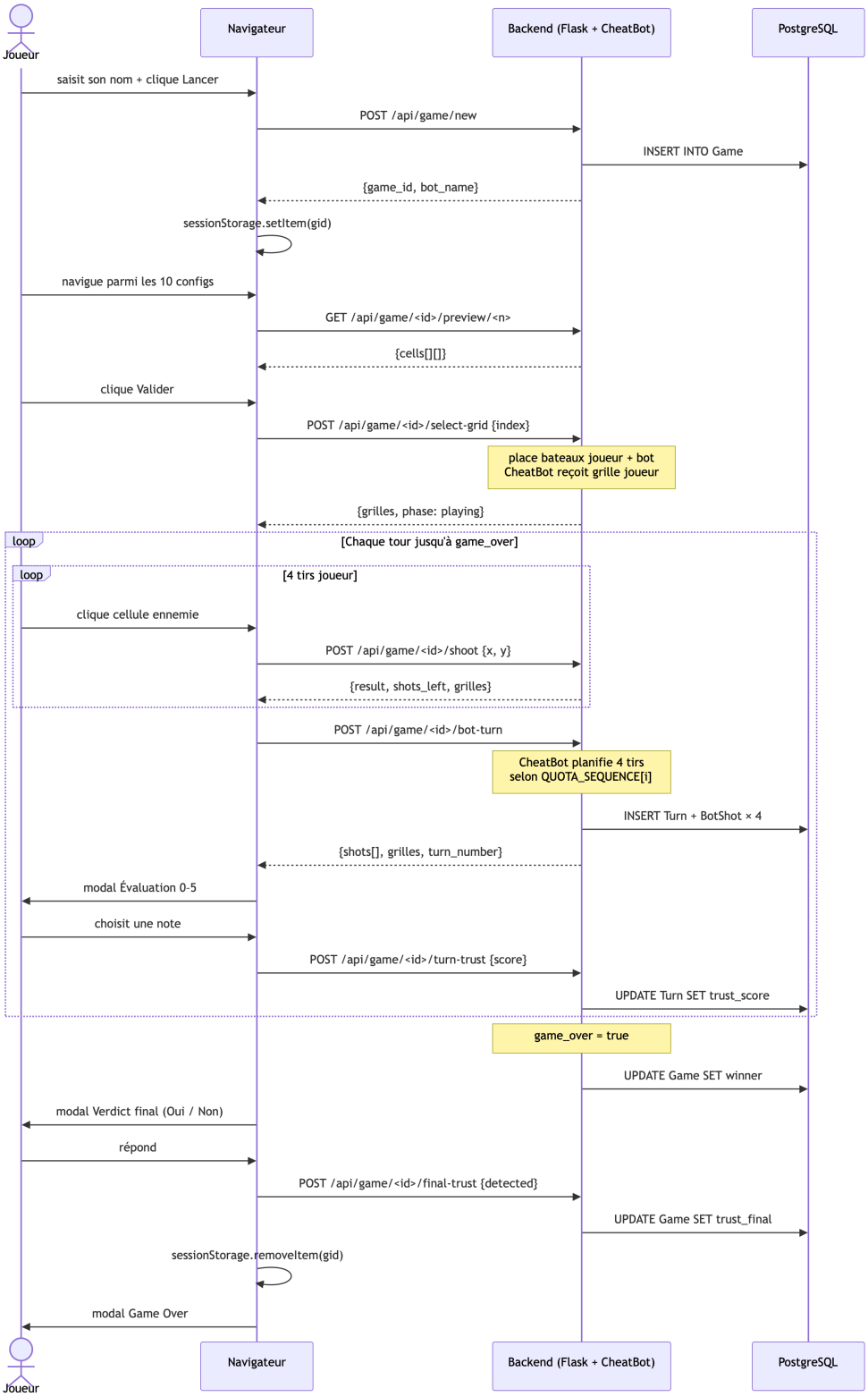


Fig. 8. – Diagramme de séquence : déroulement d'une partie

Le déroulement complet d'une session suit les étapes suivantes :

1. **Création** : le joueur saisit son nom et choisit un persona → `POST /api/game/new` → crée une `WebGame` en mémoire et une ligne `Game` en BDD
2. **Sélection de flotte** : navigation parmi 10 configurations (`GET .../preview/<idx>`) puis validation → `POST .../select-grid` → place la flotte, donne `target_grid` au bot
3. **Tour joueur** : 4 tirs successifs → `POST .../shoot` x 4 (chaque réponse met à jour l'affichage)
4. **Tour bot** : `POST .../bot-turn` → le CheatBot tire 4 fois selon le quota, résultats retournés
5. **Évaluation** : le joueur note sa suspicion 0-5 → `POST .../turn-trust`
6. Retour à l'étape 3 jusqu'à victoire ou défaite
7. **Verdict final** : le joueur répond Oui / Non → `POST .../final-trust` → `Game.trust_final` mis à jour

4.8.2 Pages et vues

Landing page (`/`) :
- Présentation de l'étude et recrutement des participants
- Statistiques dynamiques de la plateforme (récupérées via `GET /api/stats`)
- Section RGPD et garantie d'anonymat

Interface de jeu (`/game`) :
- Deux grilles 10x10 : flotte du joueur (avec tirs reçus) et grille de suivi (tirs envoyés)
- Symboles : ~ eau, `0` navire (côté joueur), `X` touché, `C` coulé, * raté
- Journal d'actions (10 dernières lignes)
- Questionnaire Likert 0-5 affiché automatiquement après chaque tour du bot
- Formulaire de verdict final en fin de partie
- Restauration de l'état après rechargement via `sessionStorage`

Dashboard admin (`/admin`) :
- Tableau des parties : joueur, date, tours joués, score Likert moyen, verdict de détection
- Tableau des joueurs : parties jouées, score moyen, taux de détection, victoires
- Graphiques : distribution Likert, corrélation quota / score, taux de détection par quota
- CRUD personnages (ajouter, activer/désactiver, supprimer)

4.8.3 Collecte des données de confiance

Deux mesures sont enregistrées en base pour chaque partie :

- **Score de confiance par tour** (`Turn.trust_score`) : échelle de Likert 0-5, saisie après chaque tour du bot : « Dans quelle mesure pensez-vous que le bot a triché ce tour ? »
- **Verdict final** (`Game.trust_final`) : binaire 0 / 1 en fin de partie : « Pensez-vous que le bot a triché durant la partie ? »

Aucune donnée personnelle n'est collectée ; l'anonymat est garanti et affiché sur la landing page conformément au RGPD.

5 Procédure d'expérimentation

5.1 Population et recrutement

La majorité de la population cible est constituée de nos amis et de membres de notre famille. La moyenne d'age est d'environ 25 ans avec une grande majorité de jeunes (autour de 23 ans). La majorité des participants sont des hommes (environ 80%).

5.2 Protocole d'expérience

5.2.1 Phase de préparation

Chaque participant est accueilli et informé de la nature générale de l'étude (détection de triche) sans révéler l'hypothèse centrale. Il leur est expliqué que les deux adversaire auront la capacité de tricher, mais sans préciser que le comportement est identique.

5.2.2 Déroulement des parties

Première partie : Le participant joue une partie contre un adversaire présenté comme HUMAIN ou IA.

Deuxième partie : Le participant joue une deuxième partie contre un adversaire présenté comme le type opposé. Les conditions de jeu restent identiques.

Pendant chaque partie, l'opposant du participant tirera selon la même séquence de quotas, garantissant un comportement rigoureusement contrôlé. La séquance est la sa suivante : 1, 2, 0, 3, 3, 2, 1, 4, 2, 1, 3 (succès garantis par tour).

Dans cette séquance, le 0 et le 4 n'apparaissent qu'une seul fois car nous faisons l'hypothèse que pour les extrèmes, les participants auront une perception plus claire de la triche (0 = pas de succès donc pas de triche, 4 = tous les tirs réussis donc triche garanti), ce qui pourrait biaiser les résultats. Nous ne souhaitions donc pas que ces extrêmes soient surreprésentés dans la séquence. De plus, il risquerait de biaiser les analyses secondaires (notamment l'effet de tour).

5.2.3 Mesure de confiance

À la fin de chaque tour, le participant doit répondre à la question suivante sur une échelle de 0 à 5 :

« Dans quelle mesure pensez-vous que votre adversaire a triché lors de ce tour ? »

Les réponses sont enregistrées immédiatement dans la base de données.

- **0** : Pas du tout triché
- **1** : Très peu probable qu'il/elle ait triché
- **2** : Peu probable qu'il/elle ait triché
- **3** : Neutre
- **4** : Probable qu'il/elle ait triché
- **5** : Certainement triché

5.2.4 Question post-expérience

À l'issue de chaques parties, une question supplémentaire est posée au participant afin de connaître son ressenti général sur la partie : *« Pensez-vous que votre adversaire a triché durant la partie ? »*

5.3 Conditions de contrôle

Pour assurer la validité interne de l'étude :

- **Équivalence des quotas** : Les deux parties utilisent des séquences identiques de réussite/échec
- **Ordre contrebalancé** : Idéalement, le type d'adversaire (humain ou IA en premier) est contrebalancé
- **Environnement standardisé** : Chaque participant réalise l'expérience dans des conditions similaires
- **Instructions standardisées** : Les consignes sont présentées de manière identique à tous

6 Résultats

6.1 Analyse descriptive

6.1.1 Caractéristiques de l'échantillon

Les caractéristiques sociodémographiques sont décrites en 5.1.1. L'analyse des résultats porte sur 20 participants (29 parties exploitables contre **Pepper Bot** (IA), **Mallory** ou **Jayson** (Humain)).

6.1.2 Scores de confiance par condition

Les évaluations de confiance (échelle 0-5, indiquant la perception de déloyauté) ont été collectées pour chaque tour de chaque partie. On peut présenter les résultats sous forme d'histogramme de la distribution des scores par condition (IA vs Humain) et par quota de réussite du bot.

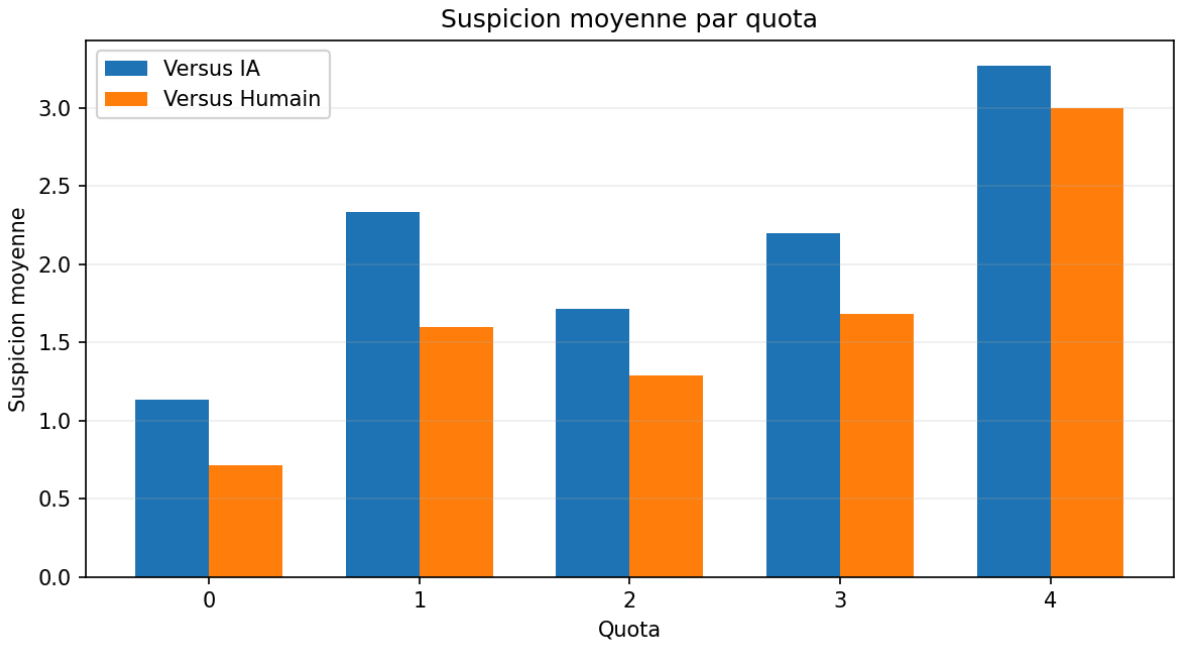


Fig. 9. – Suspicion moyenne par quota (IA vs Humain).

6.2 Analyse statistique

6.2.1 Résultats principaux

Les résultats descriptifs montrent une suspicion moyenne plus élevée lorsque l'adversaire est présenté comme une IA que lorsqu'il est présenté comme une IA. Il y a un écart systématique d'environ 0.5 point sur l'échelle de confiance, ce qui suggère un biais d'étiquetage (H_1).

6.2.2 Analyses secondaires

- **Effet d'ordre** : Comparaison entre les participants ayant affronté d'abord un humain vs l'IA
- **Effet de tour** : Évolution temporelle des perceptions au cours des parties
- **Variabilité inter-individuelle** : Identification de profils distincts

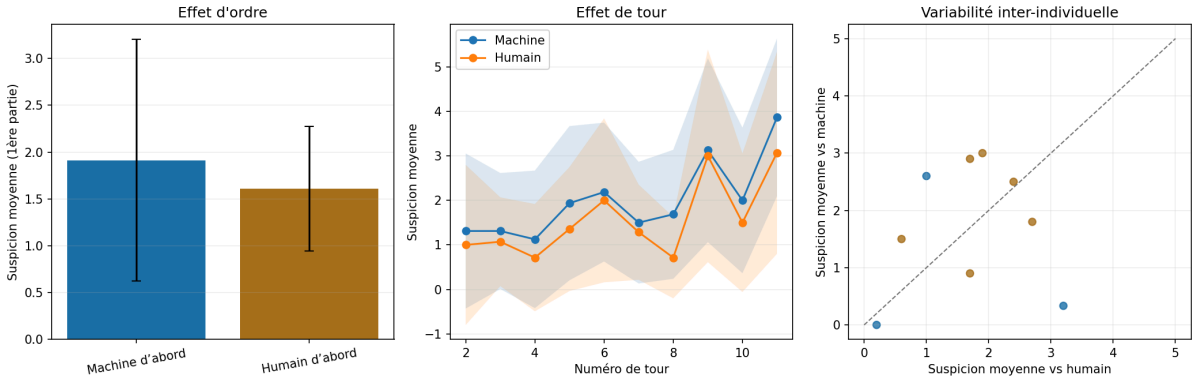


Fig. 10. – Analyses secondaires : effet d'ordre, effet de tour et variabilité inter-individuelle.

Pour les analyses secondaires, les tendances suggèrent un effet d'ordre. Les participants ayant commencé par la machine rapportent une suspicion moyenne plus forte lors de leur première partie que ceux ayant commencé par un humain (voir grpahique Effet d'ordre).

L'évolution par tour indique une hausse progressive des scores en fin de partie, avec des valeurs systématiquement plus élevées côté IA.

Enfin, la variabilité inter-individuelle reste marquée : certains joueurs déclarent une suspicion faible dans les deux conditions, tandis que d'autres perçoivent systématiquement plus de triche, ce qui suggère des profils distincts de sensibilité à la déloyauté.

Cependant, ces observations restent exploratoires en l'absence de tests statistiques formels et compte tenu de la taille limitée de l'échantillon.

6.3 Interprétation et discussion

6.3.1 Validation des hypothèses

Les résultats descriptifs sont compatibles avec $\mathbf{H_1}$: à comportement identique, l'IA est jugée plus suspecte que l'humain, ce qui va dans le sens d'un biais d'étiquetage. $\mathbf{H_2a}$ (diminution de l'écart au fil des tours) n'est pas clairement confirmée : les scores augmentent plutôt en fin de partie, avec un écart IA/humain qui reste présent. $\mathbf{H_2b}$ (effet d'ordre) est suggérée par la moyenne plus élevée chez les participants ayant commencé par la machine, mais cet effet reste exploratoire en l'absence de test statistique formel et compte tenu de la taille de l'échantillon.

6.3.2 Limitations et perspectives futures

Plusieurs limites méthodologiques doivent être signalées. L'échantillon est restreint (20 participants, 29 parties) et de convenance (proches, amis/famille), ce qui limite la représentativité et la généralisation des résultats. La population est déséquilibrée en termes de genre (≈ 80% d'hommes) et les caractéristiques sociodémographiques détaillées ne sont pas systématiquement enregistrées, empêchant des analyses fines par sous-groupes. Enfin, l'étude reste ponctuelle et ne contrôle pas totalement l'effet d'apprentissage ou d'habituation d'une partie à l'autre.

7 Conclusion

Cette étude apporte un éclairage sur la manière dont l'étiquetage « humain » vs « IA » module la perception de déloyauté pour un même comportement. Au-delà des résultats, le projet nous a appris l'importance du contrôle expérimental : pour isoler un biais perceptif, il faut garantir une stricte équivalence des stimuli (mêmes quotas, même logique d'action) et tracer finement chaque interaction en base. Le pivot technique vers une plateforme web a également montré qu'une instrumentation robuste et une UX fluide sont indispensables pour collecter des données exploitables. Enfin, l'analyse met en évidence la variabilité inter-individuelle, rappelant qu'un même comportement n'est pas perçu de manière uniforme, ce qui invite à nuancer toute conclusion globale sur la « confiance ».

Les implications dépassent le cadre ludique : ces résultats soutiennent l'idée que les jugements portés sur les agents artificiels ne dépendent pas uniquement de leur performance objective, mais aussi de leur catégorisation. Cela ouvre des pistes concrètes pour le design d'IA responsables (transparence, cadrage, communication) et pour des protocoles expérimentaux mieux outillés afin d'étudier les effets d'étiquetage en contexte réel.

Bibliographie

[1] F. Heider, *The Psychology of Interpersonal Relations*. New York: John Wiley & Sons, 1958.

[2] B. Weiner, « An Attributional Theory of Achievement Motivation and Emotion », *Psychological Review*, vol. 92, nº 4, p. 548-573, 1985, doi: 10.1037/0033-295X.92.4.548.

[3] P. Madhavan et D. A. Wiegmann, « Similarities and Differences Between Human–Human and Human–Automation Trust: An Integrative Review », *Theoretical Issues in Ergonomics Science*, vol. 8, nº 4, p. 277-301, 2007, doi: 10.1080/14639220500337708.

[4] D. Ullman, I. Leite, J. Phillips, J. Kim-Cohen, et B. Scassellati, « Smart Human, Smarter Robot: How Cheating Affects Perceptions of Social Agency », in *Proceedings of the 36th Annual Conference of the Cognitive Science Society (CogSci)*, Quebec City, Canada: Cognitive Science Society, 2014, p. 2996-3001.

[5] P. Robinette, W. Li, R. Allen, A. M. Howard, et A. R. Wagner, « Overtrust of Robots in Emergency Evacuation Scenarios », in *Proceedings of the 11th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, Christchurch, New Zealand: IEEE, 2016, p. 101-108. doi: 10.1109/HRI.2016.7451740.

[6] C. Esterwood et L. P. Robert, « Three Strikes and You Are Out!: The Impacts of Multiple Human–Robot Trust Violations and Repairs on Robot Trustworthiness », *Computers in Human Behavior*, vol. 142, p. 107658, 2023, doi: 10.1016/j.chb.2023.107658.

[7] M. Jakesch, M. French, X. Ma, J. T. Hancock, et M. Naaman, « AI-Mediated Communication: How the Perception that Profile Text was Written by AI Affects Trustworthiness », in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19)*, Glasgow, UK: ACM, 2019, p. 1-13. doi: 10.1145/3290605.3300469.

[8] A. Placani, « Anthropomorphism in AI: Hype and Fallacy », *AI and Ethics*, 2024, doi: 10.1007/s43681-024-00419-4.

[9] A. M. Lin et others, « The AI Double Standard: Humans Judge All AIs for the Actions of One ». 2024. doi: 10.48550/arXiv.2412.06040.

[10] S. S. Sebo, P. Krishnamurthi, et B. Scassellati, « ``I Don't Believe You'': Investigating the Effects of Robot Trust Violation and Repair », in *Proceedings of the 14th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, Daegu, South Korea: IEEE, 2019, p. 57-65. doi: 10.1109/HRI.2019.8673169.

[11] P. H. Kahn Jr. et others, « Human-Robot Trust and Cooperation Through a Game Theoretic Framework », in *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2016.

8 Annexes



Fig. 11. – Vidéo 1 : Premiere partie contre pepper

youtu.be/ktykt9hJhyE



Fig. 12. – Vidéo 2 : Derniere partie contre pepper

youtu.be/i_QCbBmhEOI